HYBRID AI-BASED DETECTION OF FILELESS MALWARE IN IoMT HEALTHCARE SYSTEMS

INDIVIDUAL THESIS

M.S.V. Fernando

IT22080530

Component 02: PowerShell-Based Fileless Malware Detection in Healthcare IoMT Systems

Group ID: R26-CS-006

B.Sc. (Hons) Information Technology — Cyber Security

Sri Lanka Institute of Information Technology

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: [M.S.V. Fernando]

Student ID: [IT22080530]

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Kanishka Yapa		
Co-supervisor: Mr. Dinith Primal		

ABSTRACT

PowerShell-based fileless attacks exploit legitimate scripting capabilities to execute malicious commands without relying on conventional executable files. This thesis presents an AI-driven, explainable, near-real-time detection framework designed for healthcare Internet of Medical Things (IoMT) environments. The proposed system collects Windows PowerShell Script Block Logging events (Event ID 4104), extracts script content and relevant metadata, and forwards the collected information to a FastAPI-based detection service. The detection engine employs a Term Frequency–Inverse Document Frequency (TF-IDF) vectorizer and an Extreme Gradient Boosting (XGBoost) classifier, supported by rule-based behavioral analysis to identify suspicious execution patterns, encoded commands, and other attack-associated indicators. The final cleaned dataset consists of 1,051 unique PowerShell scripts, including 693 attack-associated and 358 benign samples. During experimental evaluation, the XGBoost model achieved 97.16% classification accuracy on the reported held-out test dataset and was selected as the final machine-learning model for deployment. Behavioral indicators complement the classifier by supporting risk assessment and interpretable detection results. The implemented framework provides BENIGN and MALICIOUS classifications, risk scores, severity levels, and recommended security responses. A Streamlit-based Security Operations Centre (SOC) dashboard enables near-real-time monitoring, event visualization, alert management, and examination of detection evidence, while CSV-based storage preserves event history. Controlled functional testing demonstrated successful benign and malicious classifications through the API and dashboard workflow. However, false-positive classifications, dependence on PowerShell logging availability, and limited dataset diversity remain important challenges. The research contributes a practical and explainable XGBoost-based PowerShell fileless malware detection framework for healthcare IoMT environments, while recognising that broader real-world validation is required before operational deployment.

Keywords: PowerShell; Fileless Malware; IoMT; XGBoost; TF-IDF; Behavioral Analysis; Explainable AI; SOC Monitoring.

ACKNOWLEDGEMENTS

I acknowledge the guidance of Mr. Kanishka Yapa and Mr. Dinith Primal, and the academic and technical support provided during the research. Specific acknowledgements should be reviewed and personalised by the candidate before submission.

TABLE OF CONTENTS

Contents

ABSTRACT 3

TABLE OF CONTENTS 4

LIST OF FIGURES 8

LIST OF TABLES 9

LIST OF ABBREVIATIONS 10

1 INTRODUCTION 11

1.1 Overall project context 11

1.2 Individual component context 11

1.3 Problem statement 13

1.4 Research gap 14

1.5 Aim and objectives 14

1.6 Research questions 15

1.7 Scope, limitations and assumptions 15

1.8 Individual contribution 15

1.9 Thesis structure 16

2 LITERATURE REVIEW AND RELATED WORK 18

2.1 Review method and scope ..... **Error! Bookmark not defined.**

2.2 Fileless execution and living-off-the-land 18

2.3 Windows telemetry and Event ID 4104 19

2.4 Machine learning on script text 20

2.5 Sequence modelling ..... **Error! Bookmark not defined.**

2.6 Behavioral detection and explainability 21

2.7 Research gap synthesis 22

3 RESEARCH METHODOLOGY 23
3.1 Research design 23
3.2 Data sources and provenance 23
3.3 Dataset cleaning 25
3.4 Model representations 27
3.5 Training and validation 27
3.6 Evaluation metrics 27
3.7 Ethics and privacy 29
3.8 Reproducibility 30
4 PRODUCT AND SOLUTION DESIGN 31
4.1 Functional requirements 31
4.2 Non-functional requirements 31
4.3 Complete component architecture 32
4.4 Security architecture and trust boundaries 35
4.5 Decision logic and risk interpretation 36
4.6 Integration boundary 36
4.7 Design alternatives 37
5 IMPLEMENTATION, INTEGRATION AND TESTING 37
5.1 Development environment 38
5.2 Windows event acquisition 38
5.3 FastAPI inference service 39
5.4 Streamlit dashboard 41
5.5 Timestamp normalisation 41
5.6 Duplicate event control 41
5.7 XAI implementation 42
5.8 Controlled functional testing 42
5.9 Deployment procedure 43
6 RESULTS AND VALIDATION 44

6.1 Dataset outcome 44
6.2 Model comparison 44
6.3 Functional test outcomes 44
6.4 Operational false-positive evidence 45
6.5 Duplicate alerts and data quality 45
6.6 Time and endpoint metadata 45
6.7 Integration result interpretation 45
7 DISCUSSION 46
7.1 Interpretation 46
7.2 Why hybrid accuracy is lower 46
7.3 Clinical operational context 47
7.4 Threats to validity 47
7.5 Explainability limitations 47
7.6 Improvements 48
8 COMMERCIALISATION AND SUSTAINABILITY 49
8.1 Value proposition 49
8.2 Target users 49
8.3 Sustainability model 49
8.4 Compliance and intellectual property 49
8.5 Sustainable Development Goals 49
9 CONCLUSION AND FUTURE DIRECTIONS 50
9.1 Conclusion 50
9.2 Objective achievement 50
9.3 Individual contribution 50
9.4 Future work 50
REFERENCES 50
GLOSSARY 51
APPENDICES 51

Appendix A: Functional test specification 51
Appendix B: Operational commands 52
Appendix C: Data provenance checklist 52
Appendix D: Deployment hardening checklist 54
Deployment Security Considerations 55
Appendix E: Model artefact manifest 56
Artefact Integrity and Version Management 57
Artefact Verification Checklist 57
Appendix F: Academic integrity and AI disclosure 58
Academic Integrity and Research Responsibility 58
AI Assistance Disclosure 58
Ethical Considerations and Final Verification 59

LIST OF FIGURES

Figure 1MediRecon overall research component architeeture (author-creaied schematic).....11
Figure 2 Table 3.1: Class Distribution of the Final PowerShell Script Dataset ..... **Error! Bookmark not defined.**
Figure 3 Figure 3.1 Dataset preparation and feature representation flow 26
Figure 4.1 Complete Component 2 runtime detection architecture......34
Figure 5 Figure 4.2 Security controls and trust boundaries proposed for deployment......36
Figure 6 Figure 5.1 Inference and presentation sequence. 40

LIST OF TABLES

Table 1 List of Abbreviations and Their Definitions 10
Table 2MediRecon overall research component architeeture (author-creaied schematic) 11
Table 3 6.2: Class Distribution of the Final PowerShell Script Dataset 25
Table 4 Figure 3.1 Dataset preparation and feature representation flow 26
Table 5.1 Complete Component 2 runtime detection architecture. 34
Table 6 Figure 4.2 Security controls and trust boundaries proposed for deployment. 36
Table 7 Figure 5.1 Inference and presentation sequence. 40
Table 8 Figure 6.1 Selected live PowerShell monitoring screenshot (user-supplied operational evidence). 45
Table 9 Table A.1: Functional Test Cases and Expected Classification Results 52
Table 10 Table B.1: Operational Commands for Executing the PowerShell Detection Framework 52
Table 11 Table C.1: Dataset Provenance and Verification Checklist 54
Table 12 Recommended Deployment Security Hardening Measures for the PowerShell Detection Framework 55
Table 13 Model Artefacts and Software Components of the PowerShell Detection Framework 57
Table 14 Model Artefact Integrity and Verification Checklist 58

LIST OF ABBREVIATIONS

Abbreviation	Definition
API	Application Programming Interface
C2	Component 2
EDR	Endpoint Detection and Response
IoMT	Internet of Medical Things
SOC	Security Operations Centre
TF-IDF	Term Frequency–Inverse Document Frequency
XAI	Explainable Artificial Intelligence
EVTX	Windows Event Log file format
FP	False Positive
FN	False Negative

Table 1 *List of Abbreviations and Their Definitions*

1 INTRODUCTION

1.1 Overall project context

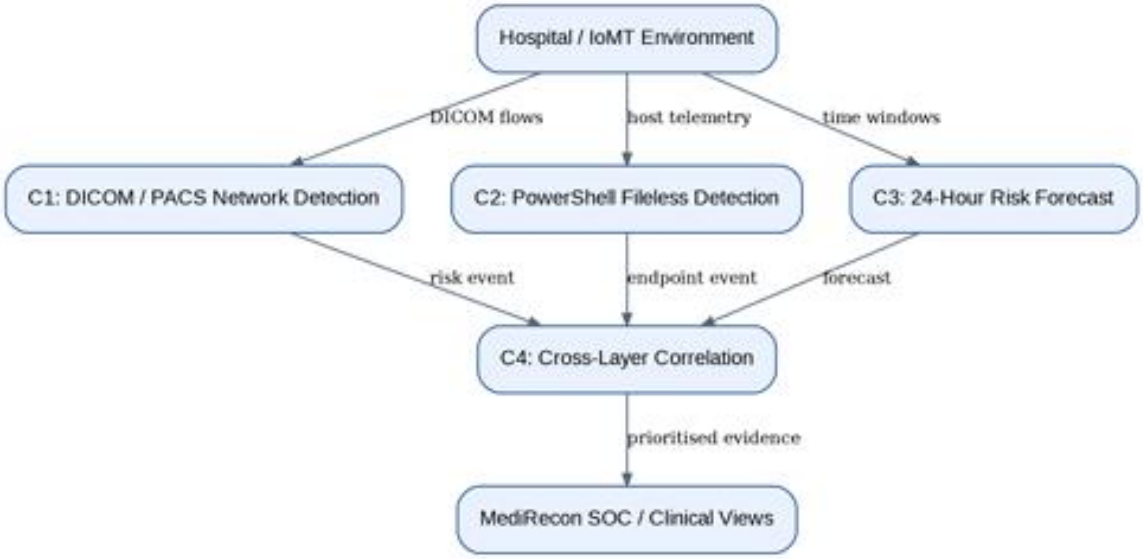


Table 2MediRecon overall research component architeeture (author-creaied schematic)

MediRecon is a group research product addressing the detection and investigation of fileless malware-related security risks in healthcare Internet of Medical Things (IoMT) environments. The overall project combines medical imaging network anomaly detection, PowerShell-based endpoint detection, next-24-hour threat forecasting and cross-layer correlation. This individual thesis focuses specifically on Component 01: DICOM Medical Imaging Network Anomaly Detection.

The individual component combines network-flow classification with medical image integrity checking. These are complementary but distinct detection functions: Random Forest processes numerical traffic descriptors, while SHA-256 compares trusted and received DICOM files. A suspicious flow does not by itself prove malware, and a matching file hash does not establish that the network was free of compromise.

For this subsection, the evidence boundary is explicit: the provided thesis template contains headings and guidance rather than primary experimental artefacts. Consequently, precise implementation assertions must be checked against the final source repository, notebook,

model metadata and logged outputs. Any quantitative claim should identify the observation window, input source, model version and method of calculation. In a hospital context, a useful security decision requires an auditable distinction among measurement, classification and response. Packet-derived features describe observed network behaviour; a classifier estimates a learned class; severity and incident response are operational policies. A correct deployment should retain all three as separately inspectable fields and should support analyst review before high-impact containment.

1.2 Individual component context

PowerShell is a legitimate Windows administrative tool widely used for system management, automation, and configuration tasks. However, attackers can misuse PowerShell to execute fileless malware without creating traditional malicious executable files, making detection challenging for conventional antivirus solutions. In healthcare Internet of Medical Things (IoMT) environments, such attacks may compromise clinical workstations, expose sensitive patient information, and disrupt essential medical services.

This individual component, “PowerShell-Based Fileless Malware in Healthcare IoMT Systems,” focuses on identifying malicious PowerShell activities using an XGBoost-based machine-learning approach supported by behavioral analysis. The framework employs Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction to transform PowerShell script content into numerical representations for XGBoost classification. Additionally, rule-based behavioral analysis identifies attack-associated indicators, including encoded commands, hidden execution, and suspicious script execution patterns.

The system collects PowerShell execution information through Windows Script Block Logging Event ID 4104 and forwards the extracted script content and relevant event metadata to a FastAPI backend for analysis. The detection engine classifies activities as BENIGN or MALICIOUS and generates risk scores, severity levels, and recommended security responses. Detection results are stored in CSV format and presented through a Streamlit-based Security Operations Centre (SOC) dashboard, enabling near-real-time monitoring, threat identification, event investigation, and security analysis.

Overall, this component strengthens healthcare endpoint security by providing an intelligent, explainable, and near-real-time PowerShell threat detection mechanism. It also generates structured security event information that can support integration with the wider MediRecon cybersecurity framework, contributing to improved visibility and coordinated threat monitoring across healthcare IoMT environments.

1.3 Problem statement

Conventional antivirus solutions primarily rely on malicious file detection and known malware signatures. However, fileless malware can exploit legitimate Windows tools such as PowerShell to execute malicious commands directly in memory, reducing the effectiveness of traditional file-based security mechanisms. This creates significant cybersecurity challenges in healthcare Internet of Medical Things (IoMT) environments, where compromised workstations may expose sensitive patient information and disrupt essential medical services.

Another major challenge is distinguishing legitimate PowerShell administrative activities from malicious execution patterns. Since PowerShell is widely used for system management and automation, detection approaches based solely on predefined signatures or suspicious keywords may generate false positives and unnecessary security alerts. This increases the workload of Security Operations Centre (SOC) analysts and can reduce the effectiveness of threat monitoring.

Furthermore, conventional endpoint monitoring approaches may provide limited behavioral context, delayed threat identification, and insufficient explanations of detection decisions. These limitations can make it difficult for security analysts to accurately assess PowerShell-related threats, investigate suspicious execution patterns, and prioritise security incidents.

Therefore, there is a need for an intelligent, explainable, and near-real-time PowerShell fileless malware detection framework that integrates machine-learning classification with behavioral analysis. The proposed component addresses this problem by employing TF-IDF-based XGBoost classification supported by rule-based behavioral indicators to analyse PowerShell script content, distinguish BENIGN and MALICIOUS activities, assess threat severity, and generate interpretable detection results. Through Windows PowerShell Event ID 4104 monitoring, FastAPI-based inference, and a centralized Streamlit SOC dashboard, the framework aims to improve endpoint threat visibility and support timely security investigation within healthcare IoMT environments.

1.4 Research gap

Existing malware detection approaches often rely on signature-based techniques, predefined behavioral rules, or standalone machine-learning models. However, these methods may struggle to identify PowerShell-based fileless malware because malicious activities can resemble legitimate administrative commands and may not leave conventional malicious executable files on disk.

Traditional text-based machine-learning approaches can identify suspicious command patterns but may produce false positives when legitimate administrative scripts contain similar features. Similarly, rule-based detection systems can recognize known malicious indicators but may fail to identify previously unseen or modified attack techniques. These limitations highlight the need for improved contextual analysis and reliable threat classification.

Furthermore, many detection approaches emphasize classification performance without adequately addressing near-real-time endpoint monitoring, explainable detection results, and integration with centralized security monitoring systems, particularly within healthcare Internet of Medical Things (IoMT) environments.

Therefore, this research investigates the integration of TF-IDF-based XGBoost classification and rule-based behavioral analysis into a unified PowerShell fileless malware detection framework. The proposed approach combines textual features and behavioral indicators with Windows Event ID 4104 monitoring, risk assessment, and explainable security outputs. Through FastAPI-based inference and a Streamlit SOC dashboard, the framework aims to improve endpoint threat visibility, support timely investigation, and provide practical security monitoring capabilities for healthcare IoMT environments.

1.5 Aim and objectives

The aim is to design, implement, and evaluate a near-real-time PowerShell fileless-malware detection component for a healthcare-oriented security platform. Objectives are to prepare labelled telemetry, train and compare classifiers, implement behavioral indicators, expose an API, ingest Event ID 4104, visualise results, and validate functional operation.

1.6 Research questions

This research investigates the effectiveness of integrating machine-learning classification and rule-based behavioral analysis for detecting PowerShell-based fileless malware in healthcare Internet of Medical Things (IoMT) environments. The following research questions guide the development and evaluation of the proposed framework.

RQ1: How effectively can TF-IDF feature extraction combined with XGBoost classify benign and attack-associated PowerShell scripts, based on accuracy, precision, recall, and F1-score?

RQ2: To what extent can rule-based behavioral analysis identify suspicious PowerShell execution patterns, including encoded commands, hidden execution, and other attack-associated indicators, to support XGBoost-based detection?

RQ3: How effectively can an integrated detection framework combining XGBoost and behavioral indicators process Windows PowerShell Event ID 4104 telemetry and generate interpretable, near-real-time security alerts with risk scores and severity levels?

RQ4: What operational limitations, false-positive classifications, and detection challenges can be identified through controlled functional testing and evaluation of the proposed system?

These research questions provide a structured foundation for evaluating the framework's classification performance, behavioral detection capabilities, operational functionality, and practical limitations. The findings will help determine the suitability of the proposed component for supporting endpoint threat detection, security monitoring, and incident investigation within healthcare IoMT environments.

1.7 Scope, limitations and assumptions

This research focuses on the development and evaluation of an AI-driven detection framework for identifying PowerShell-based fileless malware in healthcare Internet of Medical Things (IoMT) environments. The scope includes PowerShell script classification, rule-based behavioral analysis, near-real-time endpoint monitoring, risk assessment, and security alert visualization.

The proposed system utilizes Windows PowerShell Script Block Logging Event ID 4104 to collect script execution telemetry and employs TF-IDF-based XGBoost classification supported by behavioral indicators to distinguish BENIGN and MALICIOUS PowerShell activities. The behavioral analysis identifies suspicious execution characteristics, including encoded commands, hidden execution, and other attack-associated patterns. A FastAPI backend processes detection requests, while a Streamlit-based Security Operations Centre (SOC) dashboard provides centralized monitoring, detection visualization, and investigation

capabilities. Detection results are stored in CSV format to support event history and analysis. The framework is evaluated using a labelled PowerShell dataset and controlled Windows endpoint testing.

However, the research has several limitations. Detection performance depends on the quality, diversity, and representativeness of the training dataset, as well as the availability and configuration of PowerShell logging. Event ID 4104 does not capture every possible malicious activity, and legitimate administrative commands may occasionally be misclassified as malicious, resulting in false-positive alerts. Furthermore, the system primarily analyzes PowerShell script content, predefined behavioral indicators, and available endpoint metadata rather than comprehensive system behavior, memory-level malware activity, or all possible fileless attack techniques. The detection framework may also have limited effectiveness against previously unseen attack patterns that differ significantly from the training data.

The research assumes that Windows PowerShell Script Block Logging is enabled, relevant endpoint telemetry is accessible, and reliable network communication between the monitoring agent and the detection server is available. It also assumes that the trained XGBoost model and its associated TF-IDF preprocessing components are correctly loaded and available during inference. The developed component provides threat classifications, risk scores, severity levels, and recommended security responses but does not independently perform automated malware removal, endpoint isolation, or complete incident response operations.

Overall, this study is limited to a controlled experimental environment, and its findings should not be interpreted as evidence of validated performance across operational healthcare institutions. Deployment in real-world healthcare IoMT environments would require additional testing using representative clinical endpoint telemetry, privacy and data protection safeguards, performance optimization, scalability assessment, and integration validation with the wider MediRecon cybersecurity framework.

1.8 Individual contribution

The individual contribution of this research focuses on the design, development, and evaluation of an AI-driven PowerShell fileless malware detection framework for healthcare Internet of Medical Things (IoMT) environments. The component integrates XGBoost-based machine-learning classification with rule-based behavioral analysis to identify potentially malicious PowerShell activities and support endpoint security monitoring.

The research includes the preparation, cleaning, and preprocessing of a labelled PowerShell dataset containing benign and attack-associated scripts. Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction was employed to transform PowerShell script content

into numerical representations suitable for machine-learning classification. An Extreme Gradient Boosting (XGBoost) classifier was developed and evaluated to distinguish benign and malicious PowerShell execution patterns. Additionally, a rule-based behavioral detection engine was implemented to identify suspicious execution characteristics, including encoded commands, hidden execution, and potentially malicious script downloading techniques.

A FastAPI-based inference service was developed to process PowerShell scripts and generate BENIGN or MALICIOUS classifications, risk scores, severity levels, and detected behavioral indicators. Windows PowerShell Script Block Logging Event ID 4104 monitoring was incorporated to support near-real-time collection and analysis of PowerShell execution telemetry from monitored Windows endpoints. Detection results are also recorded in CSV format to maintain event history and support subsequent security investigations.

Furthermore, a Streamlit-based Security Operations Centre (SOC) dashboard was developed to visualize detection results, endpoint information, security alerts, historical events, and interpretable detection evidence. The dashboard provides centralized visibility into monitored PowerShell activities and supports security analysts in identifying and investigating potentially malicious execution patterns. The component also generates structured security information that can support integration with the broader MediRecon cybersecurity framework.

Overall, this individual contribution delivers a functional PowerShell endpoint detection component that combines TF-IDF-based XGBoost classification, behavioral threat identification, near-real-time event monitoring, risk assessment, and centralized security visualization. The developed framework contributes to improved visibility of PowerShell-based fileless threats and supports cybersecurity monitoring and incident investigation within healthcare IoMT environments.

1.9 Thesis structure

Chapter 2 reviews related methods; Chapter 3 describes data and evaluation; Chapter 4 details architecture; Chapter 5 documents implementation; Chapter 6 reports results; Chapter 7 discusses limitations; Chapter 8 examines sustainability; Chapter 9 concludes and outlines future work.

2 LITERATURE REVIEW AND RELATED WORK

This literature review examines existing research, technologies, and detection methodologies relevant to PowerShell-based fileless malware detection within healthcare Internet of Medical Things (IoMT) environments. The review focuses on living-off-the-land techniques, Windows PowerShell logging, text-based malware classification, rule-based behavioral analysis, explainable artificial intelligence (XAI), and security monitoring approaches. Particular attention is given to identifying the strengths, limitations, and practical applicability of existing solutions to the proposed XGBoost-based detection framework supported by behavioral indicators.

The review adopts a structured narrative approach, considering academic research papers, cybersecurity frameworks, technical documentation, and publicly available security studies. Relevant literature is examined to understand how legitimate Windows utilities, particularly PowerShell, can be exploited for malicious activities without relying on conventional executable malware files. Primary sources, peer-reviewed publications, and official technical documentation are prioritised to improve the reliability and academic validity of the review. Bibliographic information, publication dates, and source credibility must be verified before final submission.

A major focus of the review is the effectiveness of machine-learning approaches in identifying suspicious PowerShell execution patterns. Traditional signature-based detection methods are considered alongside text-based classification techniques, including Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction and Extreme Gradient Boosting (XGBoost). The review examines how textual features extracted from PowerShell scripts can support the classification of benign and attack-associated activities. It also considers the advantages and limitations of combining machine-learning classification with rule-based behavioral indicators to improve threat assessment and provide additional detection context.

Windows PowerShell Script Block Logging, particularly Event ID 4104, is examined as an important source of endpoint telemetry. The review considers how collected script content can support security analysis while acknowledging limitations related to logging configuration, incomplete visibility, obfuscation, and legitimate administrative activity that may resemble malicious behavior.

Explainable AI is also considered because security analysts require understandable evidence to support incident investigation. Textual feature contributions, detected behavioral indicators, risk scores, and contextual event information can assist analysts in interpreting detection outcomes. However, a high model confidence or risk score does not independently establish

malicious intent. The review therefore considers the importance of transparent detection logic and the presentation of interpretable evidence alongside machine-learning predictions.

In evaluating existing approaches, the distinction between proposed capabilities and experimentally demonstrated results is maintained. Implementation claims should be supported by reproducible evidence, including event records, saved preprocessing objects, source-code logic, model evaluation results, and controlled testing outputs.

The review further considers operational requirements such as near-real-time monitoring, API-based telemetry transmission, event persistence, and Security Operations Centre (SOC) dashboard visualization. These capabilities are relevant to healthcare environments, where security monitoring must account for sensitive information, operational continuity, and the potential consequences of incorrect classifications.

Overall, the literature review establishes the theoretical and technical foundation for the proposed AI-driven PowerShell fileless malware detection framework. It identifies the need for an approach that integrates TF-IDF-based XGBoost classification, rule-based behavioral analysis, and near-real-time endpoint monitoring while providing interpretable detection results for security analysts. The findings support the research methodology, system architecture, and evaluation criteria discussed in subsequent chapters.

2.2 Fileless execution and living-off-the-land

Fileless malware exploits legitimate operating system tools and memory-based execution techniques instead of relying primarily on traditional malicious executable files. These attacks commonly employ living-off-the-land techniques involving trusted Windows utilities, such as PowerShell and Windows Management Instrumentation (WMI), to execute malicious activities while reducing opportunities for detection by conventional file-based security solutions.

However, fileless attacks do not necessarily operate without leaving detectable evidence. PowerShell execution may generate Script Block Logging events (Event ID 4104), process creation records, and network activity logs. The effectiveness of detecting such activities depends on the availability of relevant telemetry, appropriate logging configurations, and the techniques employed by attackers to evade security monitoring.

PowerShell is frequently exploited through encoded commands, hidden execution, script obfuscation, and remote code retrieval. Since legitimate system administrators may also use similar capabilities for authorized management tasks, distinguishing benign activities from malicious execution remains challenging. Traditional signature-based detection approaches may be insufficient against previously unseen, modified, or obfuscated attack techniques.

The proposed framework addresses these challenges by employing TF-IDF-based XGBoost classification supported by rule-based behavioral analysis to identify suspicious PowerShell execution patterns and classify activities as BENIGN or MALICIOUS. The XGBoost classifier analyzes textual characteristics extracted from PowerShell scripts, while the behavioral detection mechanism identifies predefined attack-associated indicators, including encoded commands, hidden execution, and suspicious remote script retrieval patterns. The effectiveness of the framework is examined through recorded PowerShell events, model evaluation results, detection outputs, source-code evidence, and controlled functional testing.

For operational security, detection results must be interpreted alongside contextual information such as hostname, execution timestamp, event identifier, detected behavioral indicators, and calculated risk score. A high prediction probability or risk score alone does not conclusively establish malicious intent. Therefore, the framework provides structured detection information to support Security Operations Centre (SOC) analysts in investigating potentially malicious PowerShell activities within healthcare IoMT environments.

2.3 Windows telemetry and Event ID 4104

Windows PowerShell Script Block Logging provides an important source of endpoint telemetry for detecting suspicious PowerShell activities. When enabled, it records script block content through Windows Event ID 4104, allowing security analysts to examine executed PowerShell scripts and identify potentially malicious execution patterns.

Script Block Logging supports the investigation of fileless malware techniques involving encoded commands, script obfuscation, remote code retrieval, and suspicious execution behavior. Unlike traditional file-based detection, this approach provides visibility into PowerShell script content without depending entirely on malicious executable files stored on disk.

The proposed detection framework utilizes Event ID 4104 to collect PowerShell execution telemetry from monitored Windows endpoints. Relevant information, including script content, hostname, event timestamp, and record identifier, is extracted and forwarded to a FastAPI-based inference service. The collected script text is processed using Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction and analyzed through an Extreme Gradient Boosting (XGBoost) classifier supported by rule-based behavioral indicators. The detection engine generates BENIGN or MALICIOUS classifications, risk scores, severity levels, and identified behavioral indicators to support security monitoring and investigation.

However, the effectiveness of Script Block Logging depends on system configuration, event availability, and log retention policies. Disabled logging, missing event records, or incomplete

telemetry may reduce detection coverage. Additionally, collected PowerShell script content may contain sensitive information, requiring appropriate access controls, secure transmission, and data protection measures, particularly in healthcare IoMT environments.

The implementation is evaluated using available event records, source-code logic, saved TF-IDF preprocessing objects, XGBoost model outputs, and controlled functional testing results. Detection outcomes should also be interpreted alongside contextual information such as execution timestamps, hostnames, event identifiers, matched behavioral indicators, and model-generated probabilities. A high risk score alone does not conclusively establish malicious activity; therefore, the framework provides interpretable detection evidence to assist Security Operations Centre (SOC) analysts in making informed security decisions.

2.4 Machine learning on script text

TF-IDF encodes terms according to within-document frequency and corpus rarity. XGBoost can learn nonlinear relationships between numerical text features and labelled classes, but feature quality and representative sampling influence reliability.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

2.5 Behavioral detection and explainability

Explicit rules for encoded execution, hidden windows, web retrieval, and expression invocation provide analyst-readable evidence. Feature contribution methods help explain model outputs, but explanations should not be confused with causal proof.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

2.6 Research gap synthesis

Existing research on fileless malware detection has explored signature-based methods, machine-learning classification, deep-learning models, and behavioral analysis. However, translating these approaches into an integrated endpoint monitoring system remains challenging, particularly within healthcare Internet of Medical Things (IoMT) environments, where security visibility, operational reliability, and timely threat identification are essential.

A significant research gap exists in coordinating PowerShell telemetry collection, machine-learning-based classification, behavioral indicator analysis, event management, and security dashboard visualization within a unified detection workflow. Although individual machine-learning models may demonstrate strong classification performance, their practical effectiveness also depends on reliable telemetry acquisition, consistent data preprocessing, interpretable detection results, and effective integration with security monitoring systems.

The proposed research addresses this gap by integrating Windows PowerShell Script Block Logging (Event ID 4104), TF-IDF-based XGBoost classification, and rule-based behavioral analysis into an AI-driven PowerShell fileless malware detection framework. The system supports near-real-time event processing, binary BENIGN or MALICIOUS classification, risk assessment, severity determination, CSV-based event storage, and centralized Security Operations Centre (SOC) dashboard visualization.

However, strong standalone machine-learning performance does not automatically guarantee reliable detection under real-world operating conditions. Therefore, the proposed approach requires evaluation using reproducible experimental evidence, including classification metrics, saved TF-IDF preprocessing objects, XGBoost model outputs, source-code logic, recorded PowerShell events, and controlled functional testing results. The contribution of behavioral indicators to the overall risk assessment process must also be examined without assuming that their inclusion necessarily improves classification accuracy.

Furthermore, legitimate administrative PowerShell activities may resemble malicious execution patterns, increasing the possibility of false-positive classifications. Similarly, previously unseen or heavily obfuscated attack techniques may not be adequately represented in the training dataset. Detection results should therefore be interpreted using contextual information such as

hostname, execution timestamp, event identifier, detected behavioral indicators, model-generated probability, and calculated risk score.

Overall, this research contributes an integrated and interpretable PowerShell fileless malware detection workflow that combines XGBoost-based machine-learning classification, rule-based behavioral threat analysis, near-real-time endpoint monitoring, and centralized SOC visualization. The proposed framework aims to bridge the gap between experimental malware classification and practical security monitoring requirements while acknowledging the limitations of automated threat detection and the need for further validation in operational healthcare IoMT environments.

3 RESEARCH METHODOLOGY

3.1 Research design

This research adopts a design-and-evaluation methodology to develop and assess an AI-driven detection framework for PowerShell-based fileless malware in healthcare Internet of Medical Things (IoMT) environments. The research focuses on designing, implementing, and evaluating a practical endpoint security monitoring solution capable of identifying potentially malicious PowerShell activities through machine-learning classification and rule-based behavioral analysis.

The research process consists of six main stages: requirements analysis, dataset preparation, model development, system implementation, functional testing, and performance evaluation. Initially, the security challenges associated with PowerShell-based fileless malware are examined to identify the functional and operational requirements of the proposed framework. Particular attention is given to near-real-time threat detection, interpretable security results, and integration with centralized security monitoring systems.

During dataset preparation, benign and attack-associated PowerShell scripts are collected, cleaned, and labelled for binary classification. The final cleaned dataset consists of 1,051 unique PowerShell scripts, including 693 attack-associated and 358 benign samples. Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction is applied to transform the script content into numerical representations suitable for machine-learning classification. An Extreme Gradient Boosting (XGBoost) classifier is developed and evaluated to distinguish benign and malicious PowerShell execution patterns. Additionally, a rule-based behavioral analysis mechanism is implemented to identify suspicious execution characteristics, including

encoded commands, hidden execution, and potentially malicious remote script retrieval techniques.

The system implementation stage integrates Windows PowerShell Script Block Logging (Event ID 4104), endpoint telemetry collection, FastAPI-based inference, and a Streamlit Security Operations Centre (SOC) dashboard. The detection engine combines XGBoost classification outputs with predefined behavioral indicators to generate BENIGN or MALICIOUS classifications, risk scores, and severity levels. Detection results and relevant event metadata are stored in CSV format to support event history, security monitoring, and subsequent investigation.

The evaluation stage examines the classification performance of the XGBoost model using accuracy, precision, recall, and F1-score. Controlled functional testing is also conducted to assess API processing, detection outputs, event storage, and dashboard functionality. Results are examined using reproducible evidence, including saved model and preprocessing artefacts, recorded PowerShell events, source-code logic, and actual test outputs. Particular attention is given to false-positive classifications, the reliability of behavioral indicators, and the operational limitations of the implemented detection framework.

3.2 Data sources and provenance

The final labelled corpus combines existing attack-associated PowerShell telemetry with benign commands collected in a controlled Windows environment. The original malicious upstream source and its licensing should be verified from acquisition records before the thesis is submitted.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

3.3 Dataset cleaning

The final cleaned dataset consists of 1,051 unique PowerShell scripts, including 693 attack-associated and 358 benign samples. Duplicate records and entries with conflicting labels were removed during preprocessing to improve dataset consistency and reliability.

Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction is applied during preprocessing to convert PowerShell script content into numerical feature vectors suitable for the XGBoost classifier. Additionally, rule-based behavioral indicators are derived during detection to identify suspicious execution characteristics, including encoded commands, hidden execution, and remote script retrieval patterns. These features are generated through the processing pipeline rather than being stored as separate raw columns in the original dataset.

The implementation is assessed using reproducible evidence, including saved TF-IDF preprocessing objects, trained XGBoost model artefacts, source-code logic, recorded PowerShell events, and controlled functional testing outputs. This ensures that the reported system capabilities and detection results are supported by observable experimental evidence rather than design assumptions.

For operational use, detection results must be interpreted alongside relevant contextual information. Legitimate administrative PowerShell activities may resemble suspicious execution patterns, and a high model probability or calculated risk score does not independently confirm malicious intent. Therefore, the framework preserves relevant event information, including hostname, execution timestamp, event record identifier, detected behavioral indicators, and XGBoost classification evidence, to support Security Operations Centre (SOC) analysts in reviewing and investigating potentially malicious activities.

Label	Count	Share
Attack-associated	693	65.94%
Benign	358	34.06%
Total	1051	100%

Table 3 6.2: Class Distribution of the Final PowerShell Script Dataset

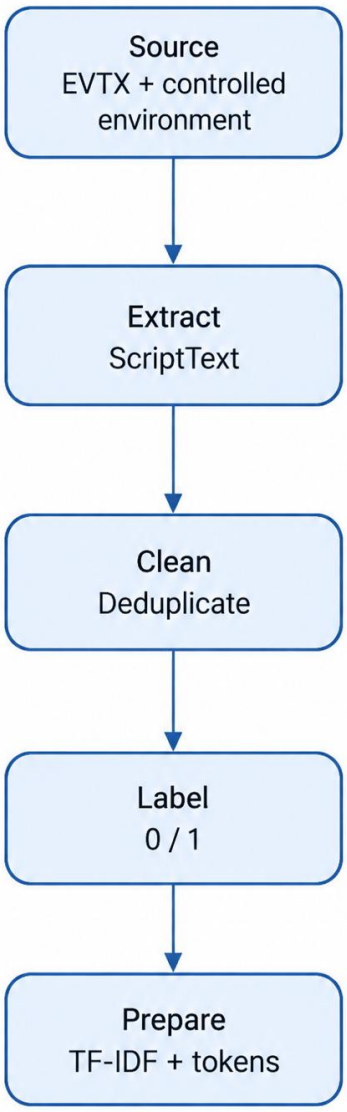


Table 4 Figure 3.1 Dataset preparation and feature representation flow

3.4 Model representations

The proposed detection framework utilizes Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction to convert PowerShell script content into numerical representations suitable for XGBoost-based classification. Unigram and bigram features are extracted to capture individual terms and consecutive term combinations within PowerShell commands. The trained XGBoost classifier processes these textual features to generate a prediction probability for distinguishing BENIGN and MALICIOUS PowerShell activities.

In addition to machine-learning classification, a rule-based behavioral analysis engine examines interpretable suspicious execution patterns, including encoded_command, hidden_window, no_profile, invoke_expression, download_string, webclient, base64_decode, and execution policy bypass. These indicators provide supplementary behavioral evidence and contribute to the overall risk assessment process. The detection engine uses the XGBoost prediction probability and identified behavioral indicators to determine the final classification, risk score, and severity level.

The implementation is assessed using reproducible evidence, including recorded PowerShell events, saved TF-IDF preprocessing objects, trained XGBoost model artefacts, source-code logic, and controlled functional testing outputs. Reported detection capabilities are considered established only when supported by corresponding experimental evidence.

For operational use, detection results must be interpreted within their execution context. Legitimate administrative PowerShell activities may contain patterns similar to malicious commands, and a high model probability or calculated risk score does not independently confirm malicious intent. Therefore, relevant information such as hostname, execution timestamp, event record identifier, matched behavioral indicators, XGBoost prediction probability, and risk assessment results should remain available to support Security Operations Centre (SOC) analyst investigation.

3.5 Training and validation

The model training and validation strategy was designed to evaluate the effectiveness of the proposed XGBoost-based PowerShell fileless malware detection framework while minimizing data leakage and ensuring reliable performance measurement. The prepared PowerShell dataset

was divided into training and testing subsets to assess the classification model using previously unseen samples.

The proposed framework employs Extreme Gradient Boosting (XGBoost) as its primary machine-learning classification model. PowerShell script content is transformed into numerical feature representations using Term Frequency–Inverse Document Frequency (TF-IDF) vectorization, incorporating unigram and bigram features. These representations are used to train the XGBoost classifier to distinguish benign and attack-associated PowerShell scripts. Additionally, a rule-based behavioral analysis engine identifies suspicious execution characteristics, including encoded commands, hidden execution, execution policy bypass, and potentially malicious remote script retrieval techniques. Behavioral indicators provide supplementary evidence during inference and support the generation of interpretable risk assessments without requiring an additional trained classification model.

To maintain evaluation integrity, TF-IDF vectorizer fitting should be performed exclusively on the training subset, with the same fitted preprocessing object subsequently applied to testing samples and live PowerShell telemetry. This approach helps prevent data leakage and ensures consistency between model training and operational inference. Duplicate or near-identical scripts across training and testing subsets may inflate performance estimates and should therefore be controlled. The final cleaned dataset contains 1,051 unique PowerShell scripts, including 693 attack-associated and 358 benign samples.

The classification performance of the XGBoost model is assessed using accuracy, precision, recall, F1-score, and confusion matrices. The documented held-out evaluation reported an accuracy of 97.16% for the standalone XGBoost classifier. This result provides evidence of the model's performance under the recorded experimental conditions but should not automatically be interpreted as the accuracy of the complete detection pipeline incorporating behavioral indicators. The operational framework is additionally assessed through controlled functional testing of API processing, classification outputs, behavioral indicator identification, risk score generation, event storage, and dashboard visualization.

The exact training and testing split ratio, random seed, XGBoost hyperparameters, TF-IDF configuration, preprocessing boundaries, and saved model provenance should be verified against the original training notebook and model artefacts to ensure reproducibility. Reported evaluation results must correspond to the specific model configuration and experimental procedure used to generate them.

Furthermore, classification outputs must be interpreted alongside supporting contextual evidence, including PowerShell script content, hostname, execution timestamp, event record

identifier, detected behavioral indicators, and XGBoost prediction probability. Legitimate administrative PowerShell commands may resemble malicious execution patterns, creating the possibility of false-positive classifications. Therefore, model predictions and calculated risk scores should support, rather than replace, security analyst investigation.

Overall, the training and validation strategy provides a structured foundation for evaluating the proposed detection framework while emphasizing reproducibility, reliable experimental evidence, transparent behavioral risk assessment, and practical Security Operations Centre (SOC) monitoring requirements within healthcare IoMT environments.

3.6 Evaluation metrics

Accuracy measures overall correct predictions; precision measures the fraction of predicted malicious scripts that are malicious in the labelled test set; recall measures detection of labelled malicious scripts; and F1 summarises the precision–recall trade-off. Confusion matrices and false-positive examples add operational context.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

3.7 Ethics and privacy

PowerShell script content can contain credentials, patient identifiers, internal hostnames, URLs, or operational secrets. Logs require access control, retention limits, sanitisation, and consent or authorisation. Controlled inert strings were used for demonstrations rather than live malware execution.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

3.8 Reproducibility

Reproduction requires the exact dataset versions, cleaning rules, split configuration, TF-IDF vectorizer, tokenizer, trained model weights, package versions, and test scripts. Artefact hashes and configuration manifests should be added from the actual project directory.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

4 PRODUCT AND SOLUTION DESIGN

4.1 Functional requirements

The component must ingest Event ID 4104, extract PowerShell script text, classify activity, calculate risk and severity, retain event metadata, display results, and expose readable indicators and model contributions.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

4.2 Non-functional requirements

Required qualities include near-real-time processing, reliable event timestamps, minimal duplicate alerts, privacy-conscious log handling, transparent outputs, graceful API failures, and repeatable local deployment.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

4.3 Complete component architecture

The proposed Component 2 architecture is designed as an AI-driven, near-real-time endpoint detection framework for identifying PowerShell-based fileless malware within healthcare Internet of Medical Things (IoMT) environments. As illustrated in Figure 4.2, the architecture follows a client-server approach that integrates remote Windows endpoint monitoring, XGBoost-based machine-learning classification, rule-based behavioral analysis, centralized event storage, and Security Operations Centre (SOC) dashboard visualization.

The detection process begins at a monitored Windows endpoint, representing a healthcare workstation or a Windows-based system operating within an IoMT environment. Windows PowerShell Script Block Logging records PowerShell script execution information through Event ID 4104. A Python-based monitoring agent installed on the endpoint retrieves relevant event records and extracts information such as script content, hostname, execution timestamp, and event record identifier.

The monitoring agent forwards the collected PowerShell telemetry to a remotely accessible FastAPI detection server through the /analyze endpoint using HTTP-based API communication. This architecture separates endpoint telemetry collection from centralized threat analysis, allowing the detection service to process PowerShell events received from monitored Windows systems. Successful remote operation depends on network connectivity, correct server configuration, and accessibility of the API endpoint.

Upon receiving an event, the detection service processes the submitted PowerShell script and transforms its textual content into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF) vectorization. The trained Extreme Gradient Boosting (XGBoost) classifier analyzes unigram and bigram features to estimate the probability of malicious PowerShell activity.

Alongside machine-learning classification, a rule-based behavioral analysis engine examines the script content for suspicious execution indicators, including encoded commands, hidden execution, execution policy bypass, command invocation patterns, and potentially malicious remote script retrieval techniques. These indicators provide additional contextual evidence for threat assessment and improve the interpretability of detection results.

The Detection and Risk Assessment Engine combines the XGBoost prediction probability with the identified behavioral indicators to determine the final BENIGN or MALICIOUS classification, calculated risk score, and severity level. This approach uses XGBoost as the deployed machine-learning classifier, while behavioral analysis operates as a complementary rule-based detection mechanism.

Following analysis, the detection service records relevant event information and classification results in CSV-based event storage, including the realtime_events.csv file. The stored records support historical event review, detection tracking, and subsequent security investigation.

The Streamlit-based SOC dashboard provides a centralized interface for visualizing PowerShell detection results, endpoint information, risk scores, severity levels, security alerts, and available behavioral evidence. Security analysts can use this interface to review recorded events and investigate potentially malicious PowerShell execution patterns without directly accessing the monitored endpoint.

The architecture also supports the preparation of structured endpoint security events for integration with the MediRecon Component 4 cross-layer correlation framework. Such integration can allow PowerShell detection information to contribute to broader security analysis alongside network and other endpoint telemetry, subject to successful API configuration and integration validation.

Although the framework is designed for near-real-time remote endpoint monitoring, its effectiveness depends on PowerShell logging availability, reliable network communication, endpoint agent operation, and detection server accessibility. Legitimate administrative commands may occasionally resemble malicious execution patterns, resulting in false-positive classifications. Therefore, contextual information such as hostname, execution timestamp, event record identifier, matched behavioral indicators, and XGBoost prediction probability is retained to support analyst investigation.

Overall, the proposed architecture provides a modular and centralized PowerShell fileless malware detection workflow by integrating remote Windows endpoint monitoring, TF-IDF-based XGBoost classification, rule-based behavioral analysis, FastAPI inference, CSV event storage, and Streamlit SOC visualization. The component is designed to contribute endpoint-level security evidence to the broader MediRecon healthcare IoMT cybersecurity framework.

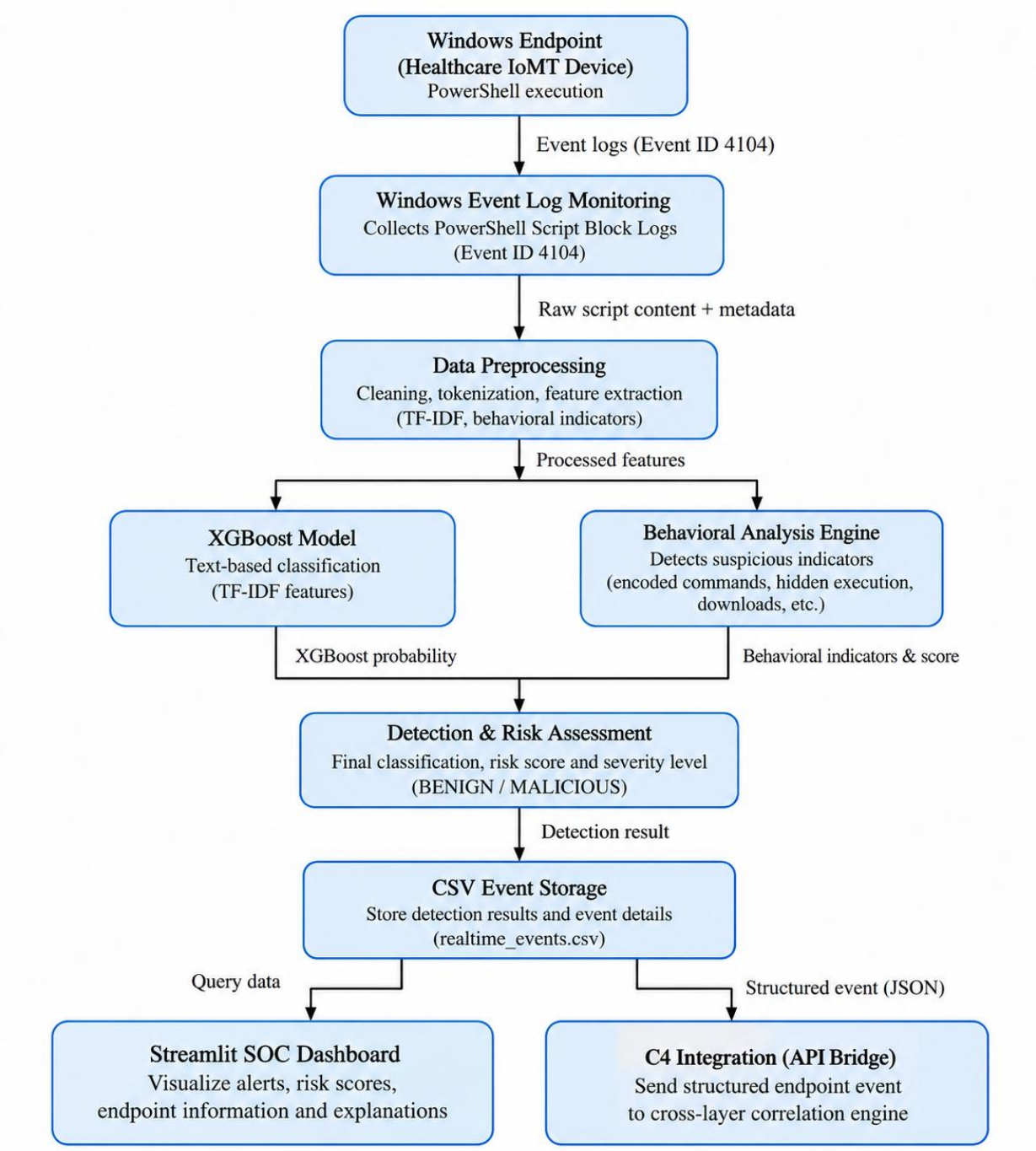


Table 5.1 Complete Component 2 runtime detection architecture.

4.4 Security architecture and trust boundaries

Windows endpoint telemetry crosses a trust boundary when sent to the inference service. The API should authenticate senders, validate request size and schema, protect event storage, and restrict dashboard access. Recommendations to quarantine or block are advisory unless a separately verified enforcement mechanism exists.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

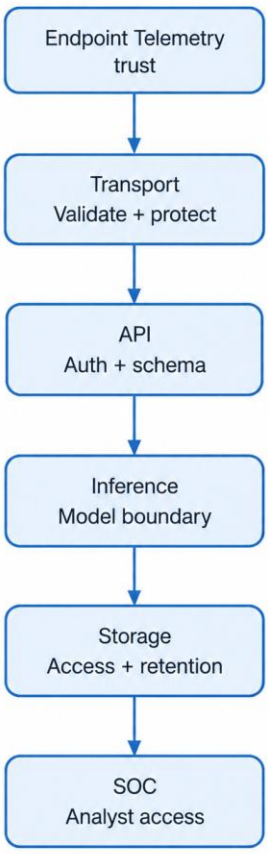


Table 6 Figure 4.2 Security controls and trust boundaries proposed for deployment.

4.5 Decision logic and risk interpretation

The interface displays detection, severity, risk score, model probabilities, and matched indicators. Exact thresholds and weighting should be extracted from the current api.py rather than reconstructed from screenshots. A model probability is not automatically a calibrated real-world attack probability.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

4.6 Integration boundary

The component can supply a structured endpoint detection signal to the wider MediRecon correlation layer. An observed HTTP status code does not by itself prove complete end-to-end correlation or mitigation; integration tests must verify event persistence and downstream interpretation.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

4.7 Design alternatives

Several detection approaches were considered during the design of the proposed PowerShell-based fileless malware detection framework, including signature-based detection, rule-based behavioral analysis, and machine-learning-based classification. Each approach offers different advantages and limitations in terms of detection capability, interpretability, computational efficiency, and operational suitability.

Signature-based detection identifies previously known malicious patterns and attack signatures. Although this approach is computationally efficient and effective against recognized threats, its ability to detect previously unseen, modified, or obfuscated PowerShell attacks may be limited. Similarly, rule-based behavioral detection identifies predefined suspicious indicators, including encoded commands, hidden execution, execution policy bypass, and remote script retrieval techniques. While these indicators provide interpretable security evidence, relying exclusively on predefined rules may result in missed detections or false-positive alerts.

Machine-learning-based classification provides an alternative approach by learning patterns from labelled PowerShell scripts. Extreme Gradient Boosting (XGBoost), combined with Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction, was selected for its ability to classify PowerShell script content using textual features. The approach supports efficient inference, structured classification outputs, and integration with endpoint security monitoring systems. The documented experimental evaluation reported an accuracy of 97.16% for the standalone XGBoost classifier, supporting its selection as the primary machine-learning model.

The final design integrates TF-IDF-based XGBoost classification with rule-based behavioral analysis to provide both machine-learning predictions and interpretable behavioral evidence. The XGBoost classifier generates a prediction probability, while the behavioral analysis engine identifies predefined attack-associated execution patterns. These outputs are used by the Detection and Risk Assessment Engine to determine the final BENIGN or MALICIOUS classification, calculated risk score, and severity level.

The implemented framework also incorporates Windows PowerShell Script Block Logging (Event ID 4104), a Python-based endpoint monitoring agent, FastAPI inference, CSV-based event storage, and Streamlit Security Operations Centre (SOC) dashboard visualization. The architecture supports centralized analysis of PowerShell telemetry and provides contextual information, including hostname, execution timestamp, event identifier, matched behavioral indicators, and classification evidence, to assist security analysts.

Although the selected approach combines machine-learning classification with behavioral analysis, the inclusion of behavioral indicators does not automatically guarantee improved detection accuracy. Legitimate administrative PowerShell activities may resemble malicious execution patterns, and previously unseen attacks may not be adequately represented in the training dataset. Therefore, detection results require contextual interpretation and further validation under realistic operational conditions.

Overall, the selected design balances classification performance, computational efficiency, behavioral interpretability, and practical endpoint monitoring requirements. By integrating XGBoost-based classification, rule-based behavioral threat identification, and centralized security visualization, the framework provides a modular and explainable approach to PowerShell fileless malware detection within healthcare IoMT environments.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development environment

The component uses Python, pandas, NumPy, scikit-learn, XGBoost, TensorFlow/Keras, FastAPI, Uvicorn, Streamlit, requests, and Windows Event Logging. Saved models and preprocessing objects are loaded by the API process.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

5.2 Windows event acquisition

PowerShell Script Block Logging was enabled and Event ID 4104 collection verified. The monitor polls approximately every two seconds, extracts script text, filters some self-generated noise, and sends structured requests for inference.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible

artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

5.3 FastAPI inference service

The FastAPI backend exposes /analyze and applies the saved vectorizer and tokenizer consistently with training. Inference returns detection, severity, risk, probabilities, indicators, and operational metadata.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

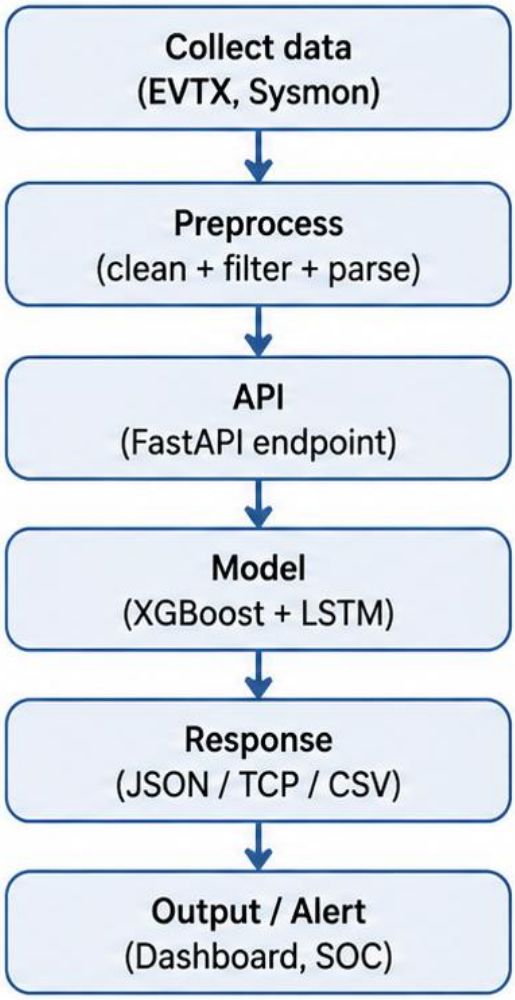


Table 7 Figure 5.1 Inference and presentation sequence.

5.4 Streamlit dashboard

The dashboard presents service status, latest detection, remote endpoint metadata, event feed, detection distribution, recent high/critical alerts, manual script analysis, SOC history, and JSON output. Refresh occurs approximately every five seconds.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

5.5 Timestamp normalisation

Event occurrence time and ingestion time must be distinct concepts. Source event timestamps should retain an explicit UTC offset or UTC representation, with Sri Lanka local time used only as a display conversion. Earlier implementations mixed naive and timezone-aware values; this is a reproducibility and auditability concern.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

5.6 Duplicate event control

The operational screenshot shows repeated record IDs and duplicated rows. A robust monitor should persist a cursor or deduplicate on host, channel, record ID, and event identity, with restart-safe handling and a documented replay policy.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

5.7 XAI implementation

Native XGBoost feature contributions provide token-level attribution. Behavioral indicators provide explicit human-readable evidence. These should be presented separately to avoid implying that one indicator explains the full model score.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

5.8 Controlled functional testing

Five defined tests included benign commands, system information queries, encoded command patterns, hidden encoded patterns, and download/execution patterns. All five expected classifications matched the observed outputs. This is a 100% pass rate for those five cases, not a 100% classifier accuracy claim.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled

test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

5.9 Deployment procedure

Start Uvicorn using python -m uvicorn api:app --host 127.0.0.1 --port 8000; then launch powershell_monitor.py; then start the dashboard with python -m streamlit run dashboard.py. Production deployment would additionally require service management, authentication, transport protection, and retention policy.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

6 RESULTS AND VALIDATION

6.1 Dataset outcome

The cleaned final dataset contains 693 attack-associated and 358 benign scripts, totalling 1,051 records. The class distribution is approximately 65.94% attack-associated and 34.06% benign.

6.2 Model comparison

The reported held-out evaluation results demonstrate that the XGBoost classifier achieved strong performance in identifying benign and attack-associated PowerShell scripts. The model was evaluated using accuracy, precision, recall, and F1-score to assess its overall classification effectiveness.

Model	Accuracy	Precision	Recall	F1
XGBoost	97.16%	97.16%	98.56%	97.86%
Probability hybrid	95.26%	98.50%	94.24%	96.32%
Behavior-enhanced hybrid	95.26%	98.50%	94.24%	96.32%

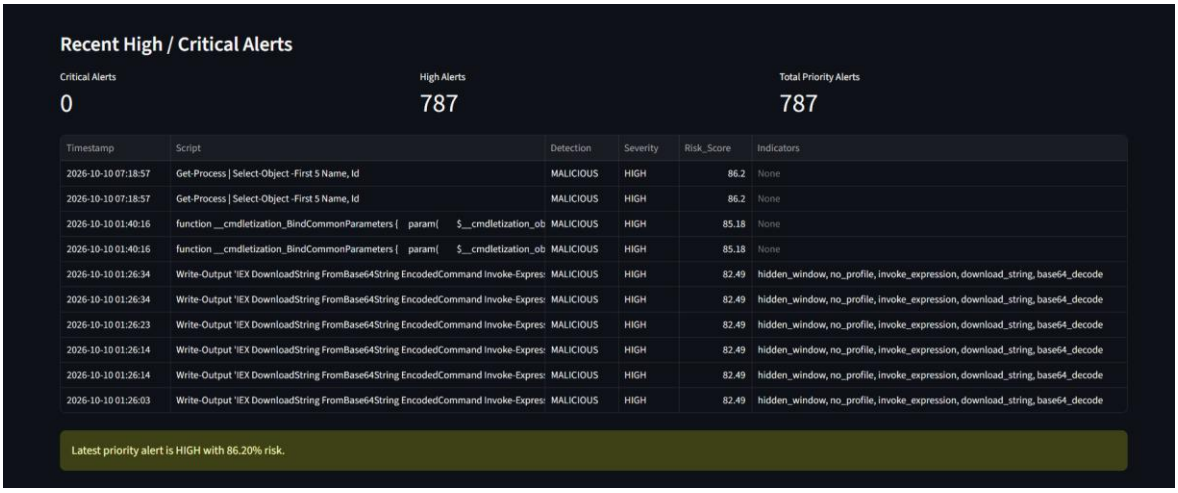
6.3 Functional test outcomes

All five controlled test cases passed. A benign Get-Date manual test yielded BENIGN, LOW, and 24.04% displayed risk, with no explicit behavioral indicators. This is useful end-to-end evidence, not a substitute for independent external validation.

Case	Expected	Observed	Result
Normal command	BENIGN	BENIGN	PASS
System information	BENIGN	BENIGN	PASS
Encoded command	MALICIOUS	MALICIOUS	PASS
Hidden encoded	MALICIOUS	MALICIOUS	PASS
Download/execution	MALICIOUS	MALICIOUS	PASS

6.4 Operational false-positive evidence

A live screenshot classified Get-Process | Select-Object -First 5 Name, Id as MALICIOUS / HIGH with 86.20% risk and no matched indicators. As a common administrative command, this warrants false-positive investigation and shows why model risk should be reviewed with contextual evidence.



Recent High / Critical Alerts

Critical Alerts	High Alerts	Total Priority Alerts
0	787	787

Timestamp	Script	Detection	Severity	Risk_Score	Indicators
2026-10-10 07:18:57	Get-Process | Select-Object -First 5 Name, Id	MALICIOUS	HIGH	86.2	None
2026-10-10 07:18:57	Get-Process | Select-Object -First 5 Name, Id	MALICIOUS	HIGH	86.2	None
2026-10-10 01:40:16	function __cmdletization_BindCommonParameters { param($__cmdletization_ob	MALICIOUS	HIGH	85.18	None
2026-10-10 01:40:16	function __cmdletization_BindCommonParameters { param($__cmdletization_ob	MALICIOUS	HIGH	85.18	None
2026-10-10 01:26:34	Write-Output 'IEX DownloadString FromBase64String EncodedCommand Invoke-Expres	MALICIOUS	HIGH	82.49	hidden_window, no_profile, invoke_expression, download_string, base64_decode
2026-10-10 01:26:34	Write-Output 'IEX DownloadString FromBase64String EncodedCommand Invoke-Expres	MALICIOUS	HIGH	82.49	hidden_window, no_profile, invoke_expression, download_string, base64_decode
2026-10-10 01:26:23	Write-Output 'IEX DownloadString FromBase64String EncodedCommand Invoke-Expres	MALICIOUS	HIGH	82.49	hidden_window, no_profile, invoke_expression, download_string, base64_decode
2026-10-10 01:26:14	Write-Output 'IEX DownloadString FromBase64String EncodedCommand Invoke-Expres	MALICIOUS	HIGH	82.49	hidden_window, no_profile, invoke_expression, download_string, base64_decode
2026-10-10 01:26:14	Write-Output 'IEX DownloadString FromBase64String EncodedCommand Invoke-Expres	MALICIOUS	HIGH	82.49	hidden_window, no_profile, invoke_expression, download_string, base64_decode
2026-10-10 01:26:03	Write-Output 'IEX DownloadString FromBase64String EncodedCommand Invoke-Expres	MALICIOUS	HIGH	82.49	hidden_window, no_profile, invoke_expression, download_string, base64_decode

Latest priority alert is HIGH with 86.20% risk.

Table 8 Figure 6.1 Selected live PowerShell monitoring screenshot (user-supplied operational evidence).

6.5 Duplicate alerts and data quality

The live feed includes repeated entries for identical PowerShell record IDs. This inflates event counts and can bias alert summaries. Deduplication should be verified before interpreting the observed 1,343 displayed monitored events as 1,343 unique executions.

6.6 Time and endpoint metadata

Screenshots show hostname, record ID, event time, and storage timestamp. Some API JSON displays UTC-offset event metadata while the dashboard shows Sri Lanka local time. These fields must be clearly distinguished and consistently normalised.

6.7 Integration result interpretation

The dashboard and API demonstrate local inference and operational presentation. The JSON screenshot includes a downstream delivery status field, but full Component 4 event consumption should be documented with corresponding receiving-side evidence before a complete integration claim is made.

7 DISCUSSION

7.1 Interpretation

Strong held-out performance demonstrates that textual patterns can be discriminative in the available labelled corpus. However, observed benign-command false positives suggest potential distribution shift, limited coverage of routine administrative scripts, or overfitting to corpus-specific token patterns.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

7.2 Why hybrid accuracy is lower

Combining classifiers can worsen aggregate accuracy if a weaker model shifts decisions across the threshold. The hybrid should therefore be justified by complementary interpretability and detection evidence rather than by unsupported claims of superior accuracy.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

7.3 Clinical operational context

Hospital security teams need explainable, low-noise alerts because unnecessary interruptions may disrupt care delivery. Script content can also be sensitive, requiring least-privilege access, safe retention, and privacy-by-design.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

7.4 Threats to validity

Threats include limited dataset diversity, uncertain upstream malicious provenance, non-independent full-dataset re-evaluation, class imbalance, uncertain temporal representativeness, test-case selection, duplicate events, and the absence of independent hospital deployment validation.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

7.5 Explainability limitations

Feature contributions identify how the model processed particular TF-IDF terms but do not prove attacker intent. Behavioral matches provide stronger literal evidence of suspicious syntax, but some such syntax may also occur in authorised administration.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

7.6 Improvements

Priorities include independent benign administration test sets, calibrated thresholds, event deduplication, event-time correctness, controlled evaluation of novel attack families, authenticated API ingestion, and cross-layer correlation with contextual evidence.

In evaluating this aspect of the component, the distinction between design intention and observed evidence is essential. The implementation should be assessed against reproducible artefacts such as event records, saved preprocessing objects, source-code logic, and controlled test outputs. An assertion is treated as established only when the corresponding artefact supports it.

For operational use, the implication is that detection output must be interpreted in context. Administrative PowerShell activity can resemble suspicious execution patterns, and a model score alone does not establish malicious intent. The associated hostname, timestamp, record identity, matched indicators, and model-specific evidence should remain available for analyst review.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition

The component offers a potential low-cost PowerShell detection signal for organisations needing visibility into administrative scripting and suspicious fileless techniques. It is a research prototype and should not be represented as a certified hospital security product.

8.2 Target users

Potential users include hospital SOC analysts, endpoint security administrators, incident responders, and academic security researchers. Adoption would require workflow integration, tuning, support, and governance.

8.3 Sustainability model

Sustainable operation depends on manageable compute demand, reproducible model updates, open documentation, alert triage, controlled data retention, and accountable human review.

8.4 Compliance and intellectual property

Before commercialisation, verify dataset licensing, model/library licences, hospital procurement requirements, privacy obligations, audit logging, and the security of any patient-related telemetry.

8.5 Sustainable Development Goals

The work aligns conceptually with SDG 3 through the resilience of healthcare services and SDG 9 through secure digital infrastructure, without asserting measured clinical outcomes.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

An operational AI-driven PowerShell fileless malware detection component was developed using Windows PowerShell Script Block Logging (Event ID 4104), TF-IDF-based XGBoost classification, rule-based behavioral analysis, FastAPI inference, CSV-based event storage, and a Streamlit SOC dashboard. The reported held-out XGBoost classification accuracy of 97.16% and five controlled functional validation tests demonstrate the feasibility of the proposed approach within the evaluated experimental environment. The framework provides BENIGN or MALICIOUS classifications, risk scores, severity levels, and interpretable behavioral evidence to support near-real-time endpoint security monitoring and investigation in healthcare IoMT environments. However, further validation using diverse real-world endpoint telemetry is required to assess generalisability, false-positive behavior, and operational reliability.

9.2 Objective achievement

The work establishes a cleaned labelled corpus, model comparison, live monitoring workflow, interpretable indicators, dashboard presentation, and controlled validation. Claims of complete production readiness or hospital-scale effectiveness remain outside the verified evidence.

9.3 Individual contribution

The individual contribution lies in integrating complementary PowerShell analysis techniques into a near-real-time, explainable endpoint detection pipeline suitable for integration with the larger research architecture.

9.4 Future work

Next steps are independent external testing, stronger benign coverage, threshold calibration, deduplication, timestamp and provenance audits, privacy controls, authenticated transport, and downstream correlation verification.

REFERENCES

Microsoft Learn, “About Logging Windows PowerShell,” Windows PowerShell documentation. [Verify URL and access date].

Microsoft Learn, “about_Script_Block_Logging,” PowerShell documentation. [Verify edition, URL and access date].

T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of KDD, 2016, pp. 785–794.

S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You?: Explaining the Predictions of Any Classifier,” KDD, 2016.

GLOSSARY

Fileless malware: malicious activity that relies substantially on in-memory execution or trusted system utilities rather than a conventional malicious executable. Script block: PowerShell code unit logged under appropriate Windows logging policy. Indicator: explicitly recognised command pattern associated with suspicious behaviour. Calibration: correspondence between model score and observed outcome frequency.

APPENDICES

Appendix A: Functional test specification

Functional testing evaluates the ability of the proposed AI-driven PowerShell fileless malware detection framework to identify benign and malicious PowerShell activities. The testing process verifies Windows PowerShell Script Block Logging (Event ID 4104) collection, TF-IDF-based XGBoost inference, rule-based behavioral indicator detection, FastAPI communication, risk score and severity generation, CSV-based event storage, and Streamlit SOC dashboard visualization. Controlled test cases are used to assess BENIGN and MALICIOUS classification outputs, system functionality, and the availability of contextual security information for analyst investigation.

Five functional test cases were defined:

Test ID	Test Scenario	Expected Output
TC-01	Normal command (Get-Date)	BENIGN
TC-02	System information query (Get-Process)	BENIGN
TC-03	Encoded command pattern	MALICIOUS
TC-04	Hidden encoded execution pattern	MALICIOUS

TC-05	Download-and-execution pattern	MALICIOUS

Table 9 Table A.1: Functional Test Cases and Expected Classification Results

The MALICIOUS expectations apply to selected attack-associated test samples, since suspicious indicators alone do not prove malicious intent.

Each test must document the test input, expected output, actual output, execution date, testing environment, and PASS/FAIL status. Results should also be verified against API responses, realtime_events.csv, and the Streamlit dashboard.

These tests support validation of the detection pipeline and identification of functional errors, false positives, and false negatives. Actual test outcomes must be recorded using experimental evidence.

Appendix B: Operational commands

The PowerShell fileless malware detection framework operates through three main components: the FastAPI inference service, PowerShell monitoring agent, and Streamlit SOC dashboard.

The following commands are used to execute the system:

Component	Execution Command
FastAPI Service	python -m uvicorn api:app --host 127.0.0.1 --port 8000
PowerShell Monitor	python powershell_monitor.py
SOC Dashboard	python -m streamlit run dashboard.py

Table 10 Table B.1: Operational Commands for Executing the PowerShell Detection Framework

The FastAPI inference service processes PowerShell telemetry using TF-IDF-based XGBoost classification supported by rule-based behavioral analysis. The Python-based monitoring agent

collects Windows PowerShell Script Block Logging events (Event ID 4104) from monitored endpoints and forwards the extracted script content and relevant metadata to the detection server. The Streamlit Security Operations Centre (SOC) dashboard presents BENIGN or MALICIOUS classifications, calculated risk scores, severity levels, detected behavioral indicators, and historical detection records to support security monitoring and investigation.

For operational testing, the FastAPI service should be started first, followed by the PowerShell monitoring agent and the Streamlit dashboard. In a two-computer environment, the FastAPI service must be configured to accept connections through an appropriate network interface, while the monitoring agent must use the detection server's reachable IP address and port. Reliable network connectivity and appropriate firewall configurations are necessary to support remote endpoint monitoring.

Controlled and harmless PowerShell commands should be used during demonstrations to verify Event ID 4104 collection, API communication, XGBoost classification, behavioral indicator detection, risk score generation, CSV-based event storage, and SOC dashboard functionality without introducing harmful activities into the test environment. These functional tests help assess the operational reliability of the framework, while quantitative detection accuracy must be established separately through labelled test data and appropriate evaluation metrics.

Appendix C: Data provenance checklist

Data provenance verification ensures the reliability, transparency, and reproducibility of the dataset used to train the proposed hybrid detection framework.

The final processed dataset, final_powershell_dataset.csv, contains **1,051 unique PowerShell scripts**, including 693 malicious and 358 benign samples. The dataset contains two primary attributes: ScriptText and Label.

The MITRE ATT&CK framework, particularly technique T1059.001 (PowerShell), provides a reference for understanding relevant attack behaviors. However, the original source of the malicious samples must be verified separately.

The following information should be documented:

Provenance Item	Required Information
Dataset Source	EVTX-to-MITRE-Attack-master
Download URL	Verified source link

Licence and Date	Usage licence and acquisition date
Data Cleaning	Duplicate removal and preprocessing procedures
Train/Test Split	Actual dataset partitioning method
Class Distribution	693 MALICIOUS and 358 BENIGN
Dataset Integrity	SHA-256 checksum of the final CSV
Label Interpretation	BENIGN or MALICIOUS classification

Table 11 Table C.1: Dataset Provenance and Verification Checklist

The original dataset sources, preprocessing code, and integrity records should be preserved to support research reproducibility. **“** https://github.com/mdecrevoisier/EVTX-to-MITRE-Attack **”**

Appendix D: Deployment hardening checklist

Deployment hardening is an important consideration for ensuring the security, reliability, and operational stability of the proposed AI-Driven Hybrid Detection Framework for PowerShell-Based Fileless Malware in Healthcare IoMT Systems. Since the framework collects PowerShell execution telemetry from Windows endpoints and transmits it to a centralized SOC monitoring system, appropriate security controls are required to protect sensitive information and prevent unauthorized access.

The following checklist outlines recommended security measures for deployment in healthcare environments.

Security Area	Recommended Security Measures
Endpoint Authentication	Authenticate endpoint monitoring agents using secure credentials or API tokens to prevent unauthorized telemetry submission and impersonation.
API Input Validation	Validate incoming JSON requests, enforce required fields, reject malformed data, and establish maximum payload sizes to prevent resource exhaustion.
Secure Communication	Implement HTTPS/TLS encryption between monitored Windows endpoints and the FastAPI inference server to protect transmitted PowerShell telemetry.
Log File Protection	Restrict access to realtime_events.csv, apply appropriate file

	permissions, and prevent unauthorized modification or deletion of detection records.
Sensitive Data Protection	Redact passwords, authentication tokens, and other sensitive information that may appear within collected PowerShell script content.
Timestamp Integrity	Preserve UTC timestamps and original Windows event identifiers to support accurate event tracking, forensic investigation, and cross-layer correlation.
Event Deduplication	Maintain persistent event tracking across monitoring-agent restarts to prevent duplicate event submissions and unnecessary SOC alerts.
Model Integrity	Document the deployed XGBoost model version, verify the integrity of saved model artefacts and associated TF-IDF vectorizer files, and implement appropriate access controls to prevent unauthorized modification, replacement, or tampering with the trained model and preprocessing components.
Dashboard Access Control	Implement authentication, role-based access restrictions, and audit logging to protect the Streamlit SOC dashboard from unauthorized access.
System Monitoring and Recovery	Monitor API availability, handle communication failures, maintain secure backups, and establish recovery procedures to support continuous security monitoring.

Table 12 Recommended Deployment Security Hardening Measures for the PowerShell Detection Framework

Deployment Security Considerations

The FastAPI inference service should accept telemetry only from authorized endpoint agents, while the monitoring agent should operate with the minimum privileges required to retrieve PowerShell events. Incoming script content must be treated as untrusted data and analyzed without executing it.

Furthermore, healthcare environments require careful handling of security telemetry because collected scripts may contain sensitive operational information. Appropriate data retention, access restrictions, and audit mechanisms should therefore be established.

The listed controls represent **recommended deployment hardening measures**, rather than confirmed features of the current research implementation. Before production deployment, these measures should be implemented and validated through security testing.

Overall, deployment hardening would improve the confidentiality, integrity, availability, and operational trustworthiness of the proposed detection framework while supporting safer integration into healthcare IoMT security infrastructures.

Appendix E: Model artefact manifest

The model artefact manifest documents the essential machine-learning models, preprocessing objects, datasets, and application files used in the proposed AI-Driven Hybrid Detection Framework for PowerShell-Based Fileless Malware in Healthcare IoMT Systems. Maintaining an accurate artefact inventory supports system reproducibility, integrity verification, version control, and future maintenance.

The proposed framework integrates TF-IDF-based XGBoost classification with rule-based behavioral analysis to identify potentially malicious PowerShell execution activities and classify them as BENIGN or MALICIOUS. The detection process also generates risk scores, severity levels, and relevant behavioral indicators to support security analysis. The following table presents the principal system artefacts and their respective responsibilities within the implemented detection framework.

Artefact Name	Description and Function
xgboost_powershell_model.pkl	Stores the trained XGBoost model used to classify PowerShell scripts based on TF-IDF features.
tfidf_vectorizer.pkl	Contains the fitted TF-IDF vectorizer used to transform script text into numerical features for XGBoost inference.
EVTX-to-MITRE-Attack-master	Contains MITRE ATT&CK-related Windows event log reference material. Its exact role in dataset preparation requires verification.
final_model_comparison.csv	Records the experimental classification performance of the XGBoost model using accuracy, precision, recall, and F1-score, supporting its selection as the primary machine-

	learning classifier in the proposed PowerShell fileless malware detection framework.
api.py	Implements the FastAPI inference service responsible for receiving telemetry and generating detection results.
dashboard.py	Implements the Streamlit SOC dashboard for visualizing classifications, risk scores, severity levels, and detection history.
powershell_monitor.py	Collects PowerShell execution telemetry from Windows endpoints and forwards it to the inference service.

Table 13 Model Artefacts and Software Components of the PowerShell Detection Framework

Artefact Integrity and Version Management

To ensure the integrity of the research implementation, each critical artefact should be verified using a SHA-256 checksum. These checksums help identify unauthorized modifications, accidental file changes, or inconsistencies between the evaluated and submitted software versions.

The manifest should record the exact filename, software version, storage location, checksum, and verification date. For directories such as EVTX-to-MITRE-Attack-master, integrity verification should use a documented file-level checksum inventory or a reproducible directory hashing procedure.

Artefact Verification Checklist

Verification Item	Required Action
File Integrity	Generate and record SHA-256 checksums.
Model Compatibility	Verify that saved models match their preprocessing objects.
Dataset Traceability	Preserve the final training dataset and source references.
Version Control	Record model and application versions.
File Availability	Confirm that all required artefacts exist in the final project directory.
Reproducibility	Verify that the saved artefacts can be loaded and used for inference.

Table 14 Model Artefact Integrity and Verification Checklist

The final submission should preserve the model artefacts, preprocessing files, source code, evaluation results, and relevant dataset documentation. Actual checksum values must be generated from the submitted files rather than assumed.

Maintaining this manifest improves the transparency, reliability, and reproducibility of the developed detection framework and supports future research, system integration, and maintenance.

Appendix F: Academic integrity and AI disclosure

Academic integrity is essential to maintaining the credibility, originality, and reliability of the proposed research, AI-Driven Hybrid Detection Framework for PowerShell-Based Fileless Malware in Healthcare IoMT Systems**.** This thesis has been prepared with AI-assisted support for academic writing, content organization, language refinement, and technical documentation.

Academic Integrity and Research Responsibility

The candidate is responsible for ensuring that all information presented in the thesis accurately reflects the research activities and actual system implementation. This includes verifying the research methodology, dataset sources, preprocessing procedures, model architectures, experimental results, and system functionalities.

All quantitative findings, including model accuracy, precision, recall, F1-score, and confusion matrices, must be supported by genuine experimental evidence. Similarly, screenshots, system architecture diagrams, code descriptions, and functional testing results must accurately represent the developed framework.

Proper academic referencing must be maintained to acknowledge external datasets, research publications, software libraries, and cybersecurity frameworks, including MITRE ATT&CK.

AI Assistance Disclosure

Artificial intelligence tools were used to assist with structuring selected thesis sections, improving academic language, and refining technical explanations. AI-generated suggestions are subject to independent verification and revision by the candidate to ensure consistency with the actual research implementation.

AI assistance does not replace the candidate's responsibility for conducting experiments, interpreting findings, validating results, or demonstrating individual research contributions.

Ethical Considerations and Final Verification

The research must follow responsible cybersecurity practices, including authorized testing, protection of sensitive endpoint telemetry, and the use of controlled environments for PowerShell execution analysis.

Before final submission, the candidate must verify all factual statements, citations, dataset provenance, experimental evidence, figures, and software artefacts. Any limitations or unverified information must be clearly acknowledged.

The use of AI assistance should be disclosed in accordance with the applicable university academic integrity policies.

Overall, this disclosure promotes transparency, research accountability, and responsible academic practice while ensuring that the final thesis represents the candidate's verified individual contribution.